

University at Buffalo SEDS

Datalogger

Software Design Document

16 October 2026

DRAFT — DESIGN BASELINE

UBSEDS-SDD-DL-001 — Version 0.1

Prepared by UB SEDS Firmware Team

Document control

Identifier	UBSEDS-SDD-DL-001, revision 0.1
Status	Proposed implementation design; firmware peer review pending.
Owner	UB SEDS Firmware Team. Reviewers are assigned through the project PR.
Source	`https://github.com/University-at-Buffalo-SEDS/training`; project `datalogger`.
Baseline	Generated working tree on 16 October 2026. This document accompanies uncommitted changes; record the merged commit in the release manifest.

Implementation status. CubeMX peripheral initialization, HAL, ThreadX, USBX CDC scaffolding, CMake configuration, and the direct-flash helper exist. Sensor drivers, three application workers, queues, durable storage, USB commands and host client described below are the intended design, not implemented or hardware-qualified behavior. Numeric budgets are initial design allocations unless explicitly labeled as build measurements.

Revision	Date	Author	Change
0.1	2026-10-16	UB SEDS	Filled project template, task and storage contracts, build baseline and verification plan.

Approval requires a firmware peer review, closure of the blocking decisions in Section 13, and recorded acceptance evidence. The original blank template remains available separately.

Contents

1 Purpose and requirements **1**

2 Context and architecture **1**

3 Drivers and platform **2**

4 Tasks, ownership, and synchronization **2**
4.1 Queue contracts . . . 2
4.2 Ownership and lock ordering . . . 3

5 Control flow and operating states **3**

6 USB command and module interfaces **4**

7 Data and persistence **5**
7.1 Command and commit sequence . . . 6
7.2 Recovery, export and deletion . . . 6

8 Firmware deployment and recovery **6**

9 Resources and timing **7**

10 Fault handling and destructive operations **7**

11 Build and deployment **8**
11.1 Getting dfu-util . . . 8

12 Verification and acceptance **9**

13 Decisions and open implementation work **10**

14 References and source baseline **10**

A Data dictionary and encoding **11**
A.1 Proposed durable slot . . . 11
A.2 Raw payload: type 1, length 32 . . . 11
A.3 Processed payload: type 2, length 48 . . . 12
A.4 RAM envelope . . . 12

List of Figures

1 Datalogger ownership and external connections. 1
2 Proposed startup sequence and readiness gate. 3
3 Operating states with storage and fault exits. 4
4 Two independent raw deliveries and verified persistence. 4

1 Purpose and requirements

Build a local flight-data training logger: acquire BMI088 acceleration/angular velocity and BMP390 pressure/temperature, calculate engineering values, persist raw and processed samples to W25Q32JV SPI flash, and export closed sessions to a computer through USB CDC. There is no network service, SEDSNet dependency, OTA transport, or application bootloader. A USB thumb drive is outside scope because this board operates as a USB device.

ID	Observable acceptance requirement
R1	One sensor worker polls both BMI088 functions and BMP390. Baseline releases are 100 Hz; barometer freshness follows its proposed 25 Hz rate.
R2	Each sample is independently offered to processing and raw storage. Both paths carry the same session, sequence and monotonic timestamp.
R3	Processing produces calibrated SI values and altitude, preserving validity and freshness.
R4	Only the data worker accesses flash at runtime; committed records have an explicit version, CRC and recovery rule.
R5	The computer can inspect status, stop a session, list and read closed sessions, and explicitly confirm deletion. Download alone never erases data.
R6	Reset rejects torn records and preserves unrelated committed sessions; interrupted deletion remains identifiable and recoverable.
R7	Full queues, storage exhaustion, device faults and USB loss are bounded, counted and reported without unbounded sensor blocking.
R8	A fresh checkout builds Debug and Release without sibling repositories. Changes use feature branches and reviewed PRs.

Timing targets for qualification: no missed 10 ms releases during a nominal 60 s run with available erased space; acquisition-start jitter at most 1 ms and processing latency at most 10 ms. These targets require the proposed 1 kHz tick and measured flash throughput. They are not claimed results.

2 Context and architecture

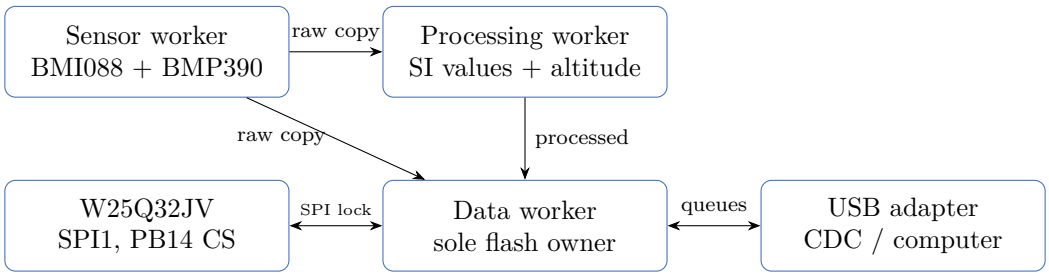


Figure 1: Datalogger ownership and external connections.

The sensor worker owns sensor initialization and acquisition. Processing owns conversions and derived outputs. Data owns session metadata, address allocation, flash operations and recovery. USB infrastructure parses host requests and exchanges bounded messages with data; it never calls a flash driver directly.

Suggested modules include `spi_bus`, `bmi088`, `bmp390`, `w25q32`, the sensor, processing and data task modules, `log_store`, and `usb_protocol`. Add their sources through the root CMake file. Generated files remain platform adapters, with calls to application initialization inside preserved user sections.

Initialize static queues, event flags, mutexes and task storage before releasing workers. Data recovers metadata before authorizing a new session. Sensor publishes only after a session ID is assigned; processing rejects malformed envelopes. Distinguish accepted-in-RAM messages from verified durable records in all status counters.

3 Drivers and platform

Item	Checked-in baseline
MCU	STM32G491CET6, Cortex-M4F, 512 KiB internal flash, 112 KiB SRAM.
Generator	CubeMX IOC targets CMake/GCC, project and file name `datalogger`. STM32CubeG4 1.6.3; X-CUBE-AZRTOS-G4 2.0.0.
Clock	Generated SYSCLK/APB clocks are 16 MHz HSI; HSI48 supplies USB. Oscillator pin assignments do not imply HSE/LSE are enabled in the clock tree.
SPI1	PA5 SCK, PA6 MISO, PA7 MOSI; master, full duplex, 8-bit, mode 0, MSB first, software NSS, divider 2: 8 MHz.
Selects	PB11 accelerometer, PB12 gyroscope, PB13 barometer, PB14 `flash_CS`. All are active low and initialize high.
USB/debug	PA11/PA12 USB device; PA13/PA14 SWD. TIM6 supplies the HAL timebase.

The schematic's STM32 and flash sheets establish the shared bus and PB14 selection. Sensor interrupt lines are unconnected, so polling is the baseline. Verify the fitted part suffix, supply, mounting axes and oscillator components against the physical board before qualification.

The SPI adapter serializes complete transactions: take an inheritance mutex, ensure other selects are high, assert one select, exchange command/address/data, deselect and unlock on every path. Keep configuration changes under the same mutex if later devices require different SPI settings. Transfers have finite timeouts; interrupt callbacks do not block on this mutex.

BMI088 initialization checks both identities, reset delays, power/measurement settings and sample scaling. BMP390 initialization loads compensation coefficients and checks data-ready/status; stale pressure remains explicitly stale. Exact range, bandwidth and oversampling settings require review and golden-vector tests. The flash driver supports bounded ID/status/read/program/sector-erase operations; it does not guess geometry after an ID mismatch.

4 Tasks, ownership, and synchronization

ThreadX uses smaller numeric priorities for higher urgency. Proposed workers use preemption threshold equal to priority and no time slice. Stacks below are byte allocations to measure, not measured maxima.

Worker	Priority	Stack	Trigger and responsibility
Sensor	5	2048 B	Absolute 10 ms deadlines; poll sensors and independently publish two raw copies.
Processing	10	4096 B	Wait on raw queue; convert/compensate, produce one result per accepted input.
Data	15	4096 B	Record/command event or flash deadline; own log allocation, persistence, recovery and deletion.
USB adapter	20	4096 B	Proposed infrastructure worker; bounded receive/transmit and command framing. Not a fourth application worker.

ThreadX timer and USBX internal workers are additional infrastructure. Inventory their actual priorities, stacks and memory pools during implementation. The generated USB application thread currently uses priority 10 and a 1024-byte stack; it is empty. Move its adapter work to the proposed budget when implementing the protocol.

4.1 Queue contracts

Each copied envelope is 64 bytes: 16 aligned 32-bit `ULONG` words. ThreadX message size is in words; backing-array size is in bytes. Compile-time assertions check both sizes. Initialize every byte; never enqueue pointers to temporary stack objects.

Queue	Depth	Bytes	Producer / consumer
`sensor_to_process`	32	2048	Sensor to processing.
`records_to_data`	128	8192	Sensor and processing to data.
`usb_commands`	8	512	USB adapter to data.
`usb_responses`	8	512	Data to USB adapter; descriptors for owned chunks.

Sensor attempts both nonblocking sends even if the first fails. Drop the new message only on the saturated path; increment separate processing-delivery, raw-log and processed-log drop counters. Sequence gaps remain visible. USB queue saturation returns BUSY when a response can be delivered; a slow host cannot block data indefinitely.

4.2 Ownership and lock ordering

Create one `spi1_mutex` with `TX_INHERIT`. Hold it only during a complete wire transaction. Never hold it across sensor conversion, flash internal busy time, queue waits, USB I/O or another lock acquisition. Data alone owns flash state and session structures, so no storage lock is needed for other workers: they send requests. A brief control lock may protect a copied status snapshot, but is never held while taking SPI.

Use static queue backing arrays and stacks, plus two 512-byte USB chunk buffers. A buffer moves FREE to DATA to USB to FREE; queue an index, length and generation, not a dangling pointer. Successful queue transfer passes ownership; failed transfer retains it. Disconnect cancels queued work and releases buffers only after active USB transfers complete or abort. No ISR allocates memory, performs blocking HAL SPI, logs synchronously, or runs program/erase operations.

Data services up to eight record steps, advances one pending flash state, then one command step before checking other inputs again. A data event flag wakes it for records or commands; a bounded timed wait also services flash deadlines. It must never wait forever on one queue while work waits on another.

5 Control flow and operating states

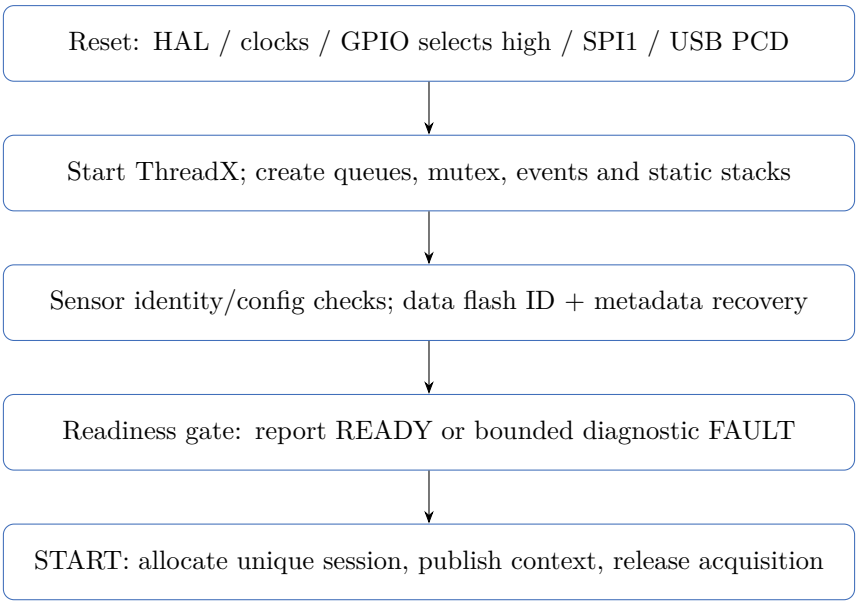
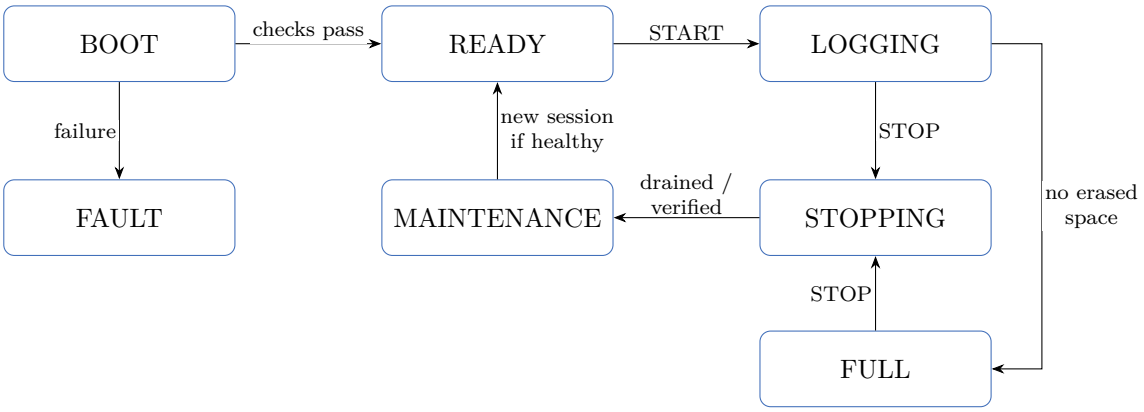


Figure 2: Proposed startup sequence and readiness gate.

BOOT performs HAL initialization and starts ThreadX. Platform object creation failures prevent logging. Data identifies flash and reconstructs session allocations. Sensor initializes devices;



Any state may enter FAULT on an unrecoverable error. FULL preserves retained sessions; STOP still drains and closes.

Figure 3: Operating states with storage and fault exits.

required-device failure prevents START. READY reports health and awaits explicit START. LOGGING creates a unique session and accepts both record types.

STOP establishes a producer cutoff. Sensor acknowledges its last sequence; processing drains all accepted earlier samples and acknowledges its last result. Only after both acknowledgments and the record queue drain does data verify pending writes and close the session. Bound the handshake to an initial 2 s budget; on timeout mark the session incomplete and report the last durable sequence rather than claiming success. Rework this budget if measured drain time requires it.

MAINTENANCE permits export and confirmed deletion of closed sessions. FULL disables additional persistence, counts rejected records and preserves retained data; acquisition and processing can continue for status. Reclaim space only through explicit maintenance deletion, then start a new session. FAULT reports its cause and prevents unsafe writes until a documented reset/recovery procedure succeeds. A USB disconnect releases leases and tokens; it does not automatically stop acquisition or delete a log.

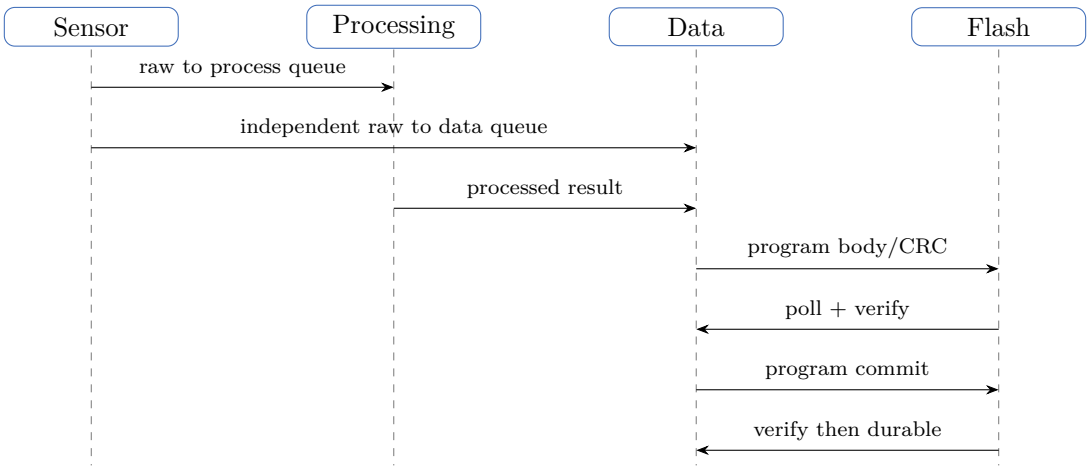


Figure 4: Two independent raw deliveries and verified persistence.

6 USB command and module interfaces

There is no network protocol. USBX CDC ACM is a byte stream to a computer; USB packets do not frame commands. Proposed requests are ASCII lines, at most 128 bytes including LF, with optional CR immediately before LF. One request is outstanding per connection. Discard malformed

or overlength input through the next LF; never execute a partial line. Parse unsigned values with explicit overflow/range checks.

Request	Contract
`STATUS`	Versions, operating state, active ID, storage usage, durable count and per-path fault/drop counters.
`LIST`	Closed session IDs, generations, committed logical lengths and incomplete flags.
`START` / `STOP`	Allocate and begin a new session / complete the cutoff-and-drain handshake.
`READ id offset count`	Read 1–512 bytes from a closed session after validating its logical bounds and export lease.
`ERASE_PREPARE id`	Validate a closed, unpinned session and issue a connection-bound confirmation token.
`ERASE_CONFIRM id generation token`	Consume a matching token once; journal deletion intent and start bounded asynchronous erasure in maintenance mode.
`OP_STATUS operation_id`	Return deletion progress, verified completion, or failure.

Replies are LF-terminated `OK ...` or `ERR code ...`. A read returns `DATA id offset length crc32` plus LF, exactly `length` binary bytes, then LF. The chunk CRC uses the algorithm in Section 7. The host respects the length even when binary bytes contain LF. Reject invalid state, unknown ID, stale generation, bounds violations and busy/full resources with distinct error codes.

READ pins the session for a proposed 30 s inactivity lease. Release the pin on explicit connection close, disconnect or lease expiry after in-flight access is quiescent. A proposed 30 s erase token binds connection, session and generation; invalidate it on disconnect, expiry or use. Pinning and token validation occur in data so deletion cannot race export. Duplicate confirmation returns existing operation status while that operation is retained, rather than erasing twice.

The host client performs STATUS, STOP, LIST, repeated READ with CRC checks, file/manifest save, then a separate explicit deletion confirmation. Reconnect resumes only after rechecking session ID and generation. The build helper is a firmware programmer, not this client.

7 Data and persistence

W25Q32JVSSIQ provides 32 Mbit: 4,194,304 bytes, inclusive addresses `0x000000--0x3FFFFF`, 256-byte pages and 4096-byte sectors. Range checks use `length <= capacity - address` after checking `address < capacity`. This external storage is accessed by SPI commands, not `HAL_FLASH_Program` or the firmware link address.

Region	Proposed allocation
`0x000000--0x000FFF`	Metadata journal A.
`0x001000--0x001FFF`	Metadata journal B.
`0x002000--0x3FFFFF`	1022 sectors for sector-aligned sessions. Reserve each session's first page for its header.

Sessions record unique monotonic IDs, generation, format, allocated sector extent, state and committed length. Generation comparison handles wrap explicitly; halt allocation rather than reuse exhausted IDs. Two journals use generation/CRC/commit checks. Verify a replacement journal before reclaiming the last valid copy. Journal allocations, session boundaries and deletion intent, not every sample. The precise metadata binary layout must be frozen and tested before implementing writes.

Records occupy 80-byte slots as defined in the data dictionary. Three slots fit one page with 16 bytes of padding. CRC-32/ISO-HDLC uses reflected polynomial `0xEDB88320`, initial and final XOR `0xFFFFFFFF`, over bytes 0–71. Check vector `123456789` gives `0xCBF43926`. Commit is four zero bytes; erased `0xFFFFFFFF` is uncommitted.

7.1 Command and commit sequence

Identify with `9F`; require the specified part's JEDEC ID `EF 40 16`. Read status with `05`: bit 0 BUSY, bit 1 WEL. Use `06` as a separate CS transaction before each modification. Read uses `03` and a three-byte address; program uses `02`; 4 KiB erase uses `20`. Addresses travel most-significant byte first, independently of little-endian record serialization.

Allocate verified erased slots only. Encode 76 bytes of body/CRC, enable writing, verify WEL, program without crossing a page, deselect and release SPI. Poll BUSY in later bounded steps. Read back and verify the body before a fresh write-enable/program of the four-byte commit; then read back the complete slot. Count a record durable only after the final verification. Split general writes at `min(remaining, 256 - (address & 255))`.

This two-program commit scheme is conditional on review of permitted partial-page programming for the fitted part. If the guarantee is inadequate, select and test a separate commit-record design before implementation. Do not silently retry over uncertain partially programmed data. Quarantine torn slots until sector erasure. Initial completion deadlines are 10 ms per program and 500 ms per sector erase, derived from the guide's datasheet maximums of 3 ms and 400 ms plus scheduling margin; validate the actual part and conditions.

7.2 Recovery, export and deletion

Boot selects the newest valid metadata journal, reconstructs allocations and accepts only versioned, committed records with valid CRC. Ignore and quarantine torn records; interrupted sessions remain identifiable as incomplete. Do not format unknown/corrupt flash automatically.

Logical export concatenates valid committed 80-byte records in append order. It omits session headers, padding and torn slots, so the physical address is not simply session base plus logical offset. An index or bounded scan maps logical offsets to slots. Data verifies a slot, copies requested bytes into an owned chunk, releases SPI, then hands it to USB.

Deletion requires maintenance mode, a closed unpinned session and a valid confirmation token. Durably journal its exclusive sector set as DELETING, erase each validated sector, blank-check in bounded chunks, then journal completion/free space. After reset, resume or report the pending deletion using that same recorded set. Never reclaim unrelated sessions or use chip erase in the normal protocol.

8 Firmware deployment and recovery

The executable links at internal flash `0x08000000`; it is programmed directly using ST-LINK or the MCU's built-in ROM DFU loader. This project contains no application bootloader, slot layout, factory-image composition, OTA package or network recovery path. Updating firmware is distinct from modifying the external log flash.

Before changing a persistent format, export retained sessions and document migration or incompatibility. A reader rejects unsupported versions without deleting them. Boot corruption produces a recoverable status/fault, not automatic erase. Flash busy timeouts and readback mismatches stop new persistence until diagnosis; repeated unbounded retries are forbidden.

Recovery procedures use a known-good firmware image and explicit hardware boot settings. Verify internal firmware identity after programming. Do not assume downloaded data is safe until the host has validated CRCs and saved the manifest. Hardware power-loss testing must cover both body/commit writes and metadata replacement/deletion boundaries.

9 Resources and timing

Budget	Design or measured value
Generated Debug build	31,452 bytes internal flash and 7,784 bytes RAM reported by linker on 16 October 2026.
Generated Release build	17,012 bytes internal flash and 7,736 bytes RAM. Both exclude the unimplemented logger design.
Application workers	10,240 bytes stacks total; USB adapter proposal adds 4096 bytes.
Four queue arrays	11,264 bytes; queue objects, control events, mutex and status structures are additional.
USB chunks	Two 512-byte buffers: 1024 bytes plus descriptors.
Flash rate	100 raw + 100 processed records/s; 80 bytes each, page padding: about 17,067 occupied bytes/s.
Ideal duration	About 245 s before metadata/session headers and other overhead. The board does not hold hours of logs at this rate.
Backlog	128 queued records absorb only 0.64 s at 200 records/s. A queue cannot cure sustained overload.

The proposed stacks, queue arrays and chunk buffers total 26,624 bytes before middleware allocations and control objects. Do not simply add this to the generated RAM figure: pool replacement may remove or change existing allocations. Inspect the final map, stack high-water marks, middleware pools and interrupt/main stacks within 112 KiB.

The generated port currently configures a 100 Hz SysTick and leaves the ThreadX tick constant at its default. The proposed design requires 1 kHz: update both the low-level reload and ThreadX tick definition consistently, then measure actual tick period and preserve the change across regeneration. A 100 Hz tick cannot establish the proposed 1 ms jitter measurement alone.

Use absolute 10 ms deadlines with wrap-safe comparisons. On overrun count it and skip to a defined future release rather than running an unlimited catch-up loop. At 8 MHz a 260-byte transaction takes 260 microseconds on the wire, excluding mutex and software overhead; measure actual worst-case lock duration. Flash internal busy waits release the bus.

Two program operations per record can consume 6 ms at the cited worst-case program time: 200 records/s can exceed sustained throughput before overhead. Qualification must justify actual rates, batching/commit changes, or a reduced logging rate. Pre-erase space in maintenance mode and preserve old sessions when full.

10 Fault handling and destructive operations

Fault	Required response
Sensor ID/read error	Bounded retries, explicit invalid/freshness flags and counters; no fabricated valid sample. Required-device initialization failure blocks START.
Queue full	Drop newest on the affected path and count it; continue other independent deliveries.
SPI failure	Deselect all slaves, release mutex, return a typed error. Never spin indefinitely holding the bus.
Flash failure/full	Stop unsafe persistence, preserve committed logs, quarantine uncertain slots and report FAULT/FULL.
USB malformed/disconnected	Discard malformed frame; invalidate leases/tokens and reclaim buffers after transfer quiescence. Logging remains independent.
Metadata ambiguity	Report recovery fault; do not guess allocations or automatically format.

The USB protocol assumes a trusted local operator and provides no authentication or confidentiality. Confirmation tokens prevent accidental or replayed deletion within the defined connection flow; they are not a security boundary against an attached malicious host. Bound lengths, numbers, command

work and timeouts before touching storage. Never expose arbitrary physical erase addresses or write-status/OTP operations through user commands.

No watchdog coverage is claimed by the generated starter. Decide whether a task-progress hardware watchdog is needed after worker implementation; if added, feed it only when required tasks make progress and verify reset behavior under injected stalls.

11 Build and deployment

The repository includes generated sources and middleware; no sibling project or runtime dependency download is needed. Install Python 3, CMake 3.22 or later, Ninja, GNU Arm GCC/G++/objcopy, and a selected flash utility on PATH. CubeCLT provides the ARM tools. Regeneration additionally requires CubeMX and the recorded firmware/middleware packs.

```
./build.py build --debug
./build.py build --release
./build.py flash --debug --method stm32prog-cli
./build.py flash --debug --method dfu
./build.py --help
```

The helper is executable and adapted from DAQ-Board with network, telemetry, simulator, bootloader and OTA paths removed. On Windows use `py -3 build.py`; Python invocation also works on other platforms. Outputs are `build/Debug_Script/datalogger.elf` and `.bin`, or the Release equivalent. CMake presets use separate `build/Debug` and `build/Release` directories.

11.1 Getting dfu-util

First run `dfu-util --version`. On macOS install Homebrew if needed, then `brew install dfu-util`. On Ubuntu/Debian run `sudo apt update` and `sudo apt install dfu-util`. On Windows extract the official Windows binary release, retain its supplied DLLs, add its executable directory to user PATH and open a new terminal. Follow the official libusb/WinUSB driver guidance for the DFU device, without replacing an unrelated device or CDC driver. See the repository README for clickable installation sources and troubleshooting.

Enter STM32 ROM DFU using BOOT0/reset and the applicable boot option settings, connect a data cable, and run `dfu-util -l` before flashing. CDC enumeration alone is not DFU mode. Confirm the internal-flash alternate setting; the helper uses alternate 0. Connect only the intended DFU target because the helper does not select by serial number. Restore normal boot selection afterward if required. Linux USB access errors require suitable device permissions/udev rules. ST-LINK remains an alternative when ROM USB DFU is unavailable.

Edit `datalogger.ioc` with project name datalogger and CMake/GCC output into the checkout root. Commit IOC and generated changes together. Keep custom logic in separate modules and preserved user sections; generated CMake is overwritten. Follow `BRANCHES.md`: start from updated main, one purpose per branch, review the diff, build both modes, open a PR, and obtain peer review before merge. Ignore build artifacts and machine-local settings.

12 Verification and acceptance

Requirement	Planned test/evidence	Status at this baseline
R8	Direct helper execution; Debug/Release configure, link and BIN generation.	Passed locally, GNU Arm 13.3.1. No board programmed.
R8	Independent source copy built from a path containing spaces, invoked outside the checkout; no sibling inputs.	Passed locally in Debug. Preset configuration and future IOC diffs remain review checks.
R1/R3	Device IDs; static and physical sensor stimuli; compensation golden vectors, stale data and tick-wrap cases.	Not implemented/tested.
R2/R7	Nominal dual delivery; independently fill each queue and verify exact per-path counters and sequence gaps.	Not implemented/tested.
R4	Logic-analyzer CS trace and maximum lock duration; only data invokes flash; bounds/page splitting tests.	Not implemented/tested.
R4/R6	CRC vector, last legal byte, bad bounds, page padding, invalid version, torn body/commit and metadata power cuts.	Not implemented/tested.
R5	STOP drain boundary; list/read/chunk CRC; malformed/partial/overlong commands, disconnect, export lease, token expiry/replay.	Not implemented/tested.
R5/R6	Reject active/pinned deletion; power cuts after intent, during erase and before completion; unrelated session preservation.	Not implemented/tested.
R7	Full-flash run, sustained throughput, 60 s timing trace, measured stack and queue high-water marks.	Not implemented/tested.

Use an explicitly allocated scratch sector for destructive flash bring-up. Identify, erase/blank-check, program a known pattern spanning pages through split commands, then compare every byte. Record board revision, firmware commit, host/tool versions, voltage conditions and analyzer captures. Never perform the scratch test on retained sessions.

Acceptance requires all mandatory requirements to have reproducible evidence, closure of storage format/throughput decisions, no unexplained nominal sequence loss, bounded error behavior, and peer approval. Generated builds establish compilation only, not correct USB enumeration, memory sizing, acquisition, persistence or recovery.

13 Decisions and open implementation work

Decision/work	Owner role	Closure criterion
Three application workers, one flash owner	Firmware lead	Implement ownership and queue contracts; peer review integration.
1 kHz tick and timing	Platform developer	Update port reload and tick constant together; measured tick/jitter and regeneration check.
USB memory and adapter	USB developer	Replace generated 1024-byte pools with measured sufficient budgets; successful USBX initialization and bounded I/O.
Sensor configuration	Driver developer	Freeze range, bandwidth, oversampling, axis transform and calibration; golden vectors and physical tests.
Metadata binary format	Storage developer	Versioned journal/session layout with power-cut tests and no ID reuse.
Partial-page commit guarantee	Storage developer	Verify fitted-part constraints or select an alternative durable commit scheme before programming records.
Rate and duration	Firmware lead/operator	Approve required run duration and sustained measured throughput; adjust rates/format if necessary.
USB client and protocol	USB developer	Implement framing, ownership, STOP, export and deletion tests.
Hardware qualification	Test owner	Record all Section 12 evidence and reviewer sign-off.

Role assignments are responsibilities, not claims that individuals have accepted work. Assign people and track each item through issues/PRs. An optional Kalman filter is deferred; add its state model, noise assumptions, initialization and validation as a separate reviewed feature if requested.

The checked-in USBX memory pool is currently 1024 bytes, while the generated application requests a 1024-byte USB memory region and an additional 1024-byte thread stack plus allocator overhead. It is insufficient as a runtime design and must be resized before claiming working USB. This document does not silently treat the generated values as qualified allocations.

14 References and source baseline

The following local files accompany the training repository. Manufacturer copyright and licenses remain with their owners. Consult the complete documents while implementing drivers; the design does not replace their timing tables or register descriptions.

1. `CrashNBurn-Project-Guide.pdf`, training baseline, 15 September 2026. Source for the three-worker design, queue proposals, record format, recovery and USB workflow; especially Sections 1, 4 and 5.2–5.5.

2. `Schematics.pdf`, six sheets: STM32 sheet 2 and external flash sheet 5 establish SPI1 and PB14; sensor sheets establish connected signals.
3. `datasheets/W25Q32JV-SPI-Flash.pdf`, Winbond W25Q32JV, Rev K, April 2026. Viewer pages 17, 22–28, 36, 38 and 66: suffix/status, commands, page program, erase and timing. Page numbers are one greater than printed Winbond numbers.
4. `datasheets/BMI088-IMU.pdf`, Bosch BST-BMI088-DS000-19, revision 1.9, January 2024: initialization, data interfaces, measurement configuration and conversion.
5. `datasheets/BMP390-Barometer.pdf`, Bosch BST-BMP390-DS002-07, revision 1.7, March 2021: configuration, compensation, data-ready and interface behavior.
6. `datasheets/STM32G491-MCU.pdf`, ST DS13122 Rev 4, April 2024: CET6/LQFP48 pinout and electrical limits. The datasheets README records download sources and hashes.
7. `datalogger.ioc`, generated Core, AZURE_RTOS, USBX, CMake and linker files, working tree dated 16 October 2026: current implemented configuration. Record the merged revision at release.
8. The original UB SEDS Overleaf template supplies the document structure and branding. RF-Board is a documentation example cited by the training guide; its networking and boot architecture do not apply here.
9. dfu-util official project and releases: `https://dfu-util.sourceforge.net/` and `https://dfu-util.sourceforge.net/releases/`; Homebrew formula `https://formulae.brew.sh/formula/dfu-util`; Ubuntu package `https://packages.ubuntu.com/dfu-util`. Installation guidance checked 16 October 2026.

A Data dictionary and encoding

A.1 Proposed durable slot

All integer fields are explicitly serialized little-endian. Floating values use IEEE 754 binary32 with documented units. Never write native C structs directly. A decoder checks length, type, format, reserved bytes, CRC and commit before exposing a record.

Offset	Bytes	Field	Meaning
0	4	Magic	Bytes `44 4C 4F 47` (DLOG).
4	1	Version	1 for the proposed format.
5	1	Type	1 raw sensor, 2 processed.
6	2	Payload length	0–48; type-specific expected length checked.
8	4	Session ID	Unique persisted ID; never reused after reset.
12	4	Input sequence	Acquisition sequence shared by both record types.
16	8	Timestamp	Monotonic microseconds, relative to session start. Resolution follows the clock source; 64-bit storage does not imply microsecond precision.
24	48	Payload	Type-specific encoding below; unused bytes zero.
72	4	CRC32	CRC-32/ISO-HDLC of bytes 0–71.
76	4	Commit	Zero only after verified body; erased word is uncommitted.

A.2 Raw payload: type 1, length 32

Offsets below are relative to the payload, not the whole slot. Signed integers use two's complement. Session metadata records actual sensor configuration and scaling.

Offset	Encoding	Meaning
0	`u32`	Valid/fresh flags.
4	3 times `i16`	BMI088 acceleration counts, sensor axes x/y/z.
10	3 times `i16`	BMI088 gyro counts, sensor axes x/y/z.
16	`u32`	BMP390 pressure ADC, unsigned 24-bit value in low bits.
20	`u32`	BMP390 temperature ADC, unsigned 24-bit value in low bits.
24	`u32`	Age of last fresh barometer reading in microseconds.
28	`u32`	Acquisition error/status bitmask.

BMI088 settings and BMP390 compensation coefficients must be saved in versioned session metadata so raw data can be reproduced offline. Unavailable channels are zero-filled with validity clear, not inferred valid from their numeric value.

A.3 Processed payload: type 2, length 48

Offset	Encoding	Meaning
0	`u32`	Valid/fresh flags inherited or derived from inputs.
4	3 times `f32`	Calibrated body-axis acceleration in m/s^2.
16	3 times `f32`	Calibrated body-axis angular rate in rad/s.
28	`f32`	Compensated pressure in Pa.
32	`f32`	Compensated temperature in degrees Celsius.
36	`f32`	Barometric altitude in m relative to stored reference pressure.
40	`u32`	Barometer age in microseconds.
44	`u32`	Processing error/status mask.

Flag bits 0–3 mean accel/gyro/pressure/temperature valid; bits 4–7 mean each corresponding channel fresh; bit 8 means altitude valid. Other flag bits are reserved zero. Raw records keep altitude-valid clear. Status bit 0 means a read error, bit 1 stale barometer, bit 2 invalid compensation, bit 3 out-of-range conversion; reserve other bits. Derived non-finite values clear validity. Freeze any additional codes through a versioned protocol/format change.

Processing uses recorded mounting transform, bias/scale and reference pressure. Initial altitude is the standard-atmosphere relation $h = 44330[1-(p/p_0)^{0.1903}]$ m with positive finite pressures; it is an approximation, not a navigation guarantee. Raw and processed pairs correlate by session/sequence even if their append order differs.

A.4 RAM envelope

A 64-byte queue envelope contains record type and schema version (one byte each), payload length (two bytes), input sequence (four), timestamp (eight) and 48 payload bytes. The session ID is an immutable active-session context shared before producers start; the STOP barrier ensures no old-session envelopes remain before a new context is published. This preserves all 48 processed payload bytes without truncation. Data adds the session ID when encoding the durable slot. The type/version/length header determines the payload decoder.

Control and USB queues use separate tagged 64-byte schemas. Control messages carry operation/session/generation IDs and bounded arguments; responses carry status, buffer index, length and CRC. No dynamic pointer or host-supplied physical flash address is accepted. Freeze these wire-independent control encodings and assertions with the application implementation.